Temple University
Japan Campus

Individual Project Proposal

Hand Music

Student: Anastasia Sasaki
Proposal type: New Software System

CIS 3296 — Software Design

1. Project Summary and Motivation

Musical instruments and DAWs (digital audio workstations) can require significant practice with the interface, so having a software where you can perform your idea quicker and in a visually interactive way will be an interactive way to work with music. The proposed software is this: a real-time application that turns hand gestures [1] into music using a webcam. It will use hand tracking, a trained gesture classifier and a configurable gesture-to-action mapping for making gestures into musical actions / MIDI chords. A MIDI output will then be sent to a synthesizer, where that music can be heard. On the screen, there will be a live on-screen overlay showing the detected gesture and the chord / volume for it. The intended users will be those who already know basic music theory such as chord names and basic harmony. The design will be such that the software will be accurate, with all the gestures creating the intended sounds, and the sounds that the user wishes to create will be customizable so they may change which gestures map to which chords through an editable mapping file. The value of the project is that it will be low cost, hardware free and a visual way to perform and experiment with music.

2. Background and Related Work

Earlier hand-tracking work often needed special hardware such as depth sensors [2]. MediaPipe Hands removes this need: it finds the hand in the image and predicts 21 landmarks from a single RGB camera [2]. Google's current Hand Landmarker task packages this and works with a live camera feed [1]. Their gesture dataset came from 30 people with limited backgrounds [2], so tracking quality may vary with lighting and hand angle. This project uses this tracking instead of training its own vision model, and treats tracking quality as a risk to test.

Because the tracker outputs coordinates, gestures can be classified directly from landmarks. The simplest way is to check whether each finger is bent or straight and map that combination to a gesture [2]. Sung et al. used this with a rule-based classifier, which is easier to create and extend without training data, and a neural network, which is more accurate for borderline cases [3]. Uboweja et al. let users add their own gestures by recording a few webcam images per gesture [4].

Gestop is the most similar system to this project. It sends MediaPipe keypoints to a gesture recognizer, then looks up the matching action in a human-readable JSON file that users can edit. New gestures can be added by recording samples and retraining [5]. It reports high accuracy for static gestures and dynamic gestures, and describe latency only as not noticeable [5]. However, it controls desktop actions rather than music.

For music, CAVI uses an armband worn on the performer's forearm [6], while *Gulf* uses a webcam with MediaPipe so a pianist can control live electronics with hand gestures [7]. *Gulf* is a specific performance setup and does not report accuracy or latency. DAWs can also be hard for beginners [8]. In a study with 20 participants, a standard interface had higher usability and efficiency, while a voice- and face-controlled one promoted more engagement and creativity [8]. A new interface should therefore not give up reliability for engagement, so this project aims for gestures that are simple and easy to tell apart.

None of the works reviewed here combines webcam hand tracking with chord and volume control for music. This project combines these parts: interchangeable rule-based and trained recognizers, an editable mapping from gestures to chords, volume and pitch range from the other hand, and MIDI output to a synthesizer. Latency matters more for music than for desktop control, because a delay between a gesture and the sound is directly audible.

3. Implementation Goals and Scope

3.1. Implementation Goals

1. **Goal 1.** Track both of the user's hands from a webcam using MediaPipe, and tell the left and right hand apart.
2. **Goal 2.** Recognize a set of left-hand gestures with a trained classifier, with a simple rule based version to compare against, and map each gesture to a chord.
3. **Goal 3.** Set the volume from the height of the right hand, and the pitch range from the direction of its palm.
4. **Goal 4.** Send the chord, pitch range, and volume as MIDI to a synthesizer, using an editable mapping file for the chords.
5. **Goal 5.** Show the detected gesture, chord, and volume on screen.

3.2. Core Functional Requirements

- **R1.** The system shall read live video from a webcam and detect the landmarks of both hands in real time, labeling each as left or right.
- **R2.** The system shall classify the left-hand pose into one of the supported gestures, or into no gesture when none is shown.
- **R3.** The system shall read the gesture-to-chord mapping from an editable file, so the user can change which gesture plays which chord without changing the code.
- **R4.** The system shall set the volume from the vertical position of the right hand (lower is quieter, higher is louder) and choose the pitch range from the direction of the right palm.
- **R5.** The system shall send MIDI messages for the chosen chord, pitch range, and volume to a synthesizer, and stop the chord when the left-hand gesture ends.
- **R6.** The system shall show a live overlay with the detected gesture, chord, and volume.

3.3. Initial Scope

In scope. A desktop application that uses one webcam and both hands. The left hand makes a small set of static gestures that map to chords through an editable file. The right hand controls volume by height and pitch range by palm direction. The system sends MIDI to an existing synthesizer and shows the on-screen overlay.

Out of scope. Some features that could be added: dynamic gestures, a built-in sound synthesizer, an in-app tool for recording and training new gestures, mobile support.

4. Initial Design

4.1. Use-Case Diagram

The system has two actors, shown in figure 1. The *Performer* is the person playing music with their hands in front of a webcam. The *Synthesizer* is the outside program that receives MIDI and makes the sound. The Performer can play chords with the left hand, control volume with the right hand, and change the pitch range with the right palm. The Performer can also edit the

gesture-to-chord mapping and watch the live overlay to see what the system detected. The first three goals all send MIDI to the Synthesizer, so they share one included use case for that.

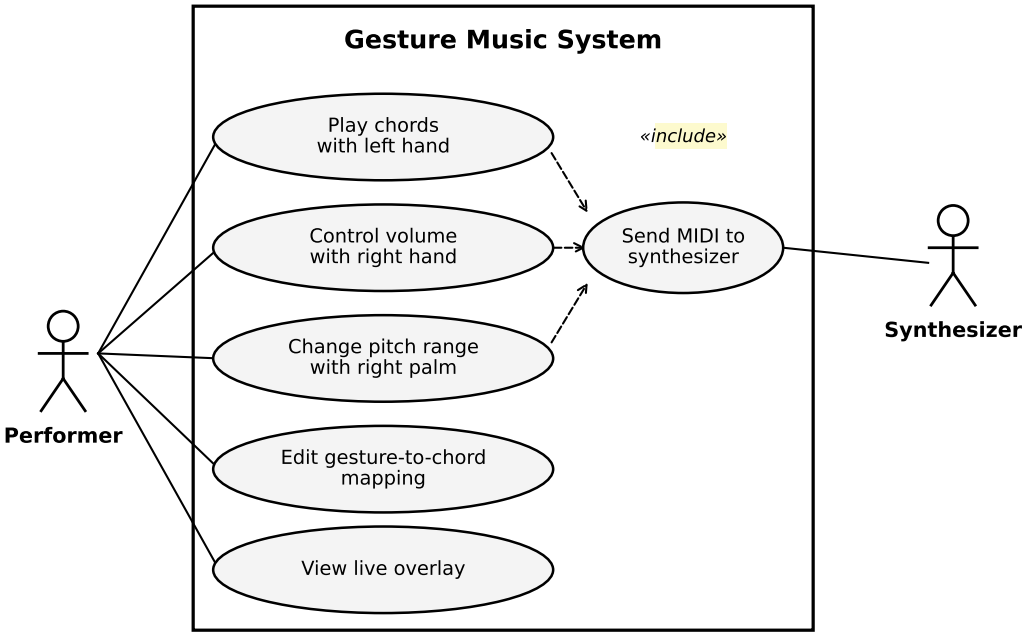


Figure 1: Initial use-case diagram for the proposed system.

4.2. Class Diagram

figure 2 shows the main classes. `GestureMusicApp` owns one of each main part and, for every webcam frame, runs `processFrame()`. `HandTracker` wraps MediaPipe and returns up to two `HandData` objects, one for each hand. The left hand goes to a `GestureClassifier`, an interface with a rule-based and a trained implementation, and `ChordMapping` looks up the chord for the detected gesture. The right hand goes to `RightHandInterpreter`, which turns its height into a volume and its palm direction into a pitch range. `MidiOutput` sends the result to the synthesizer, and `OverlayView` shows it on screen.

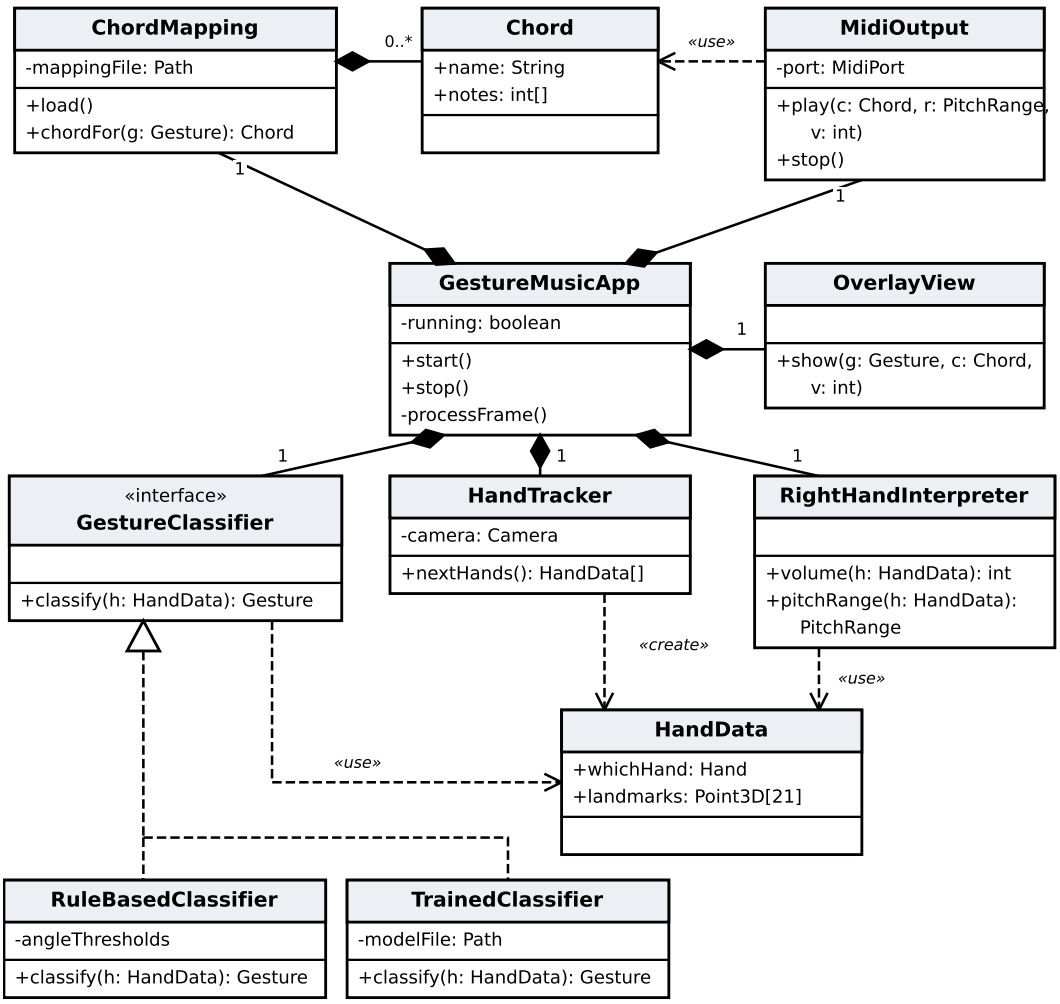


Figure 2: Initial UML class diagram for the proposed system.

5. Feasibility and Supporting Evidence

The system is planned as a Python desktop application that runs on a laptop with its built-in webcam. Hand tracking uses MediaPipe's Hand Landmarker [1], which reads a live camera feed and returns hand landmarks and which hand each one is. The camera frames are read with OpenCV. The left-hand classifier will be a small neural network trained on landmarks recorded by the author, next to a simple rule-based version. Chords will be sent as MIDI messages to an existing software synthesizer, so no sound engine needs to be built.

The project looks feasible for three reasons. First, hand tracking, the hardest part, already exists and runs in real time on ordinary devices [2]. Second, the classifier is small: Gestop recorded about 2000 samples per gesture in a couple of minutes and reports no noticeable latency [5], and a rule-based version needs no training data [3]. Third, the scope is limited to static left-hand gestures and an existing synthesizer.

The biggest problem is the right hand. Volume from hand height should be simple, but MediaPipe gives only a relative depth, so palm direction may be less reliable. Figure 3 shows the existing tracking component that the project builds on. It is not a result from this project, and testing the right-hand controls on the author's own webcam is the first planned step.

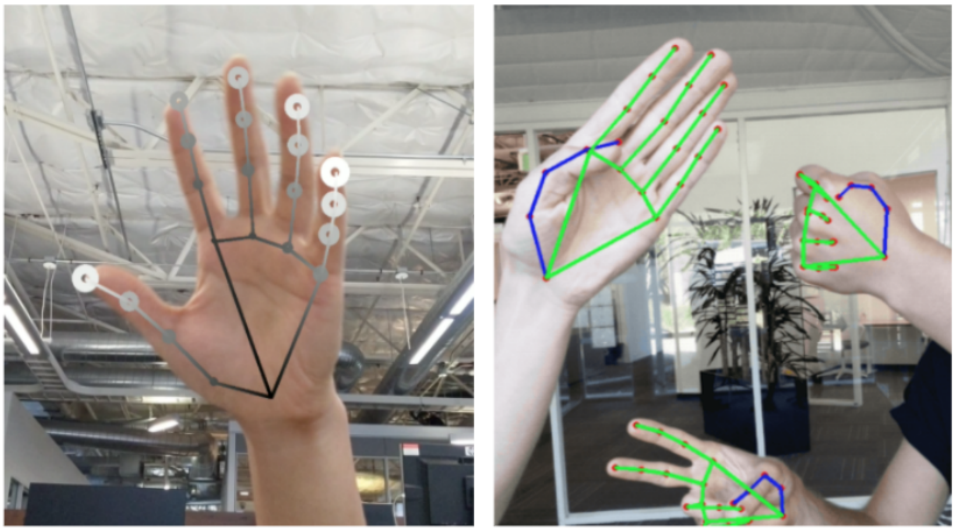


Figure 1: Rendered hand tracking result. (Left): Hand landmarks with relative depth presented in different shades. The lighter and larger the circle, the closer the landmark is towards the camera. (Right): Real-time multi-hand tracking on Pixel 3.

Figure 3: Existing component: MediaPipe Hands. Left: hand landmarks with relative depth. Right: real-time tracking of several hands on a phone. Adapted from [2]; not a result from this project.

6. Resources and Risks

Table 1: Initial project risks and mitigation strategies.

Risk / Constraint	Why it matters	Mitigation / Fallback
Palm direction may be unreliable	MediaPipe gives only relative depth, so the pitch range could change by accident.	Test it first. If it is too noisy, control the pitch range with a simpler signal, such as a second left-hand gesture.
Left and right hands may be swapped	A mirrored camera image can flip the labels, so chords and volume would be controlled by the wrong hand.	Show the detected hand on the overlay and keep a single setting to flip the image.
Gestures may be misrecognized	A wrong gesture plays the wrong chord. Tracking quality may vary with lighting and background.	Use a small set of clearly different gestures, add a "no gesture" class as Gestop does, and record training samples in different lighting.
Delay between gesture and sound	A delay is directly audible in a musical instrument.	Keep the per-frame work small, lower the camera resolution if needed, and check the delay by ear early in development.

References

[1] Google, *Hand landmarks detection guide*, Google AI Edge, MediaPipe Solutions, Accessed: Oct. 14, 2026, 2026. [Online]. Available: https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker.

[2] F. Zhang, V. Bazarevsky, A. Vakunov, *et al.*, "MediaPipe Hands: On-device real-time hand tracking," in *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, arXiv:2006.10214, Seattle, WA, USA, 2020.

[3] G. Sung, K. Sokal, E. Uboweja, *et al.*, "On-device real-time hand gesture recognition," *arXiv preprint arXiv:2111.00038*, 2021.

[4] E. Uboweja, D. Tian, Q. Wang, *et al.*, "On-device real-time custom hand gesture recognition," in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, 2023, pp. 4273–4277. DOI: 10.1109/ICCVW60793.2023.00461.

[5] Sriram S K and N. Sinha, "GESTOP: Customizable gesture control of computer systems," in *Proc. 8th ACM IKDD CODS and 26th COMAD (CODS COMAD 2021)*, Bangalore, India: ACM, 2021. DOI: 10.1145/3430984.3430993.

[6] C. Erdem, B. Wallace, and A. Refsum Jensenius, "CAVI: A coadaptive audiovisual instrument–composition," in *Proc. Int. Conf. New Interfaces for Musical Expression (NIME)*, Auckland, New Zealand, 2022. DOI: 10.21428/92fbeb44.803c24dd.

[7] Z. Kanga, R. Gleave, and T. J. Mitchell, "Gulf: Piano and gesture-controlled live electronics," in *Music Proc. Int. Conf. New Interfaces for Musical Expression (NIME '26)*, London, UK, 2026. DOI: 10.5281/zenodo.20782166.

[8] P. Buccellato, A. Bianco, and C. Rottondi, "Soundy: A multimodal audio interface for educational music production," *IEEE Access*, vol. 13, pp. 153 105–153 122, 2025. DOI: 10.1109/ACCESS.2025.3604631.